

计算机网络

传输层

考试大纲

五、传输层

(一) 传输层提供的服务

1. 传输层的功能
2. 传输层寻址与端口
3. 无连接服务与面向连接服务

(二) UDP协议

1. UDP数据报
2. UDP校验

(三) TCP协议

1. TCP段
2. TCP连接管理
3. TCP可靠传输
4. TCP流量控制
5. TCP拥塞控制

1.1 传输层的功能概述

传输层向上面的应用层提供通信服务，属于面向通信部分的最顶层，同时也是用户功能中的最底层。

- **核心任务：** 为运行在不同主机上的进程之间提供逻辑通信。
- **端到端通信：** 网络层提供主机之间的逻辑通信，而传输层提供进程之间的逻辑通信。
- **屏蔽底层细节：** 传输层向高层用户屏蔽了下面网络核心的细节，使应用进程看见的就好像是在两个传输层实体之间有一条端到端的逻辑通信信道。



传输层的三大核心功能



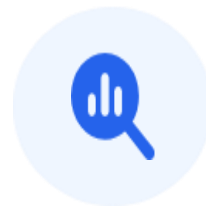
进程间逻辑通信

打破了网络层仅提供主机到主机的通信限制，实现了真正的应用到应用的端到端通信。



复用与分用

复用是指发送方多个应用进程可使用同一个传输层协议；分用是指接收方传输层剥去报文首部后能把数据正确交付给目的应用进程。



差错检测

网络层只检查IP数据报的首部，而不检验数据部分。传输层则会对收到的报文进行首部和数据部分的全面差错检测。

网络层 vs 传输层

网络层 (Network Layer)

提供 **主机之间** 的逻辑通信。

路由器只工作在下三层（物理层、数据链路层、网络层）。网络层尽最大努力交付，不保证可靠性。

寻址标识：**IP地址** 具体的主机)。

传输层 (Transport Layer)

提供 **进程之间** 的端到端逻辑通信。

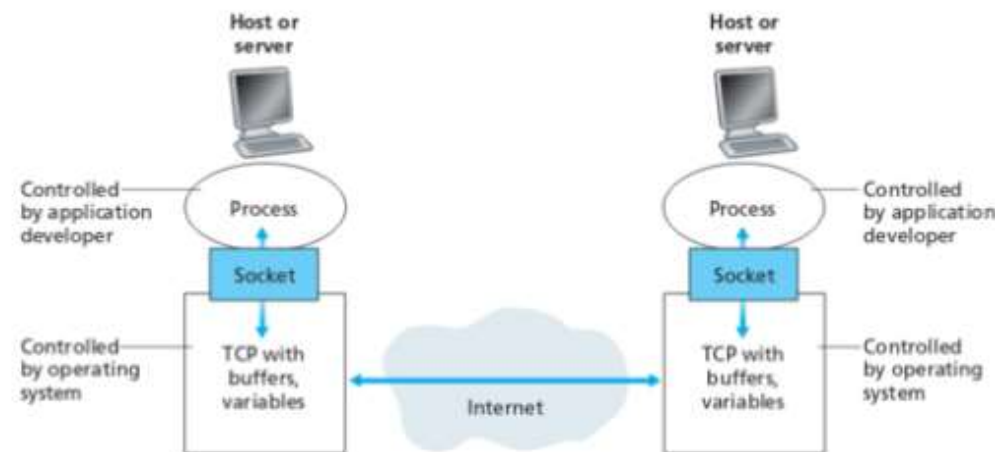
传输层协议仅运行在端系统（主机）中。路由器不处理传输层报文。

寻址标识：**端口号 (Port)** 的具体进程)。

1.2 传输层寻址与端口

由于进程的创建和撤销是动态的，传输层不能直接使用进程PID来标识。因此，引入了**协议端口号（Port）**，简称端口。

- **软件端口：** 传输层所使用的端口是软件端口，是应用层各种协议进程与传输实体进行层间交互的一种地址。
- **硬件端口：** 不要与路由器或交换机上的硬件接口（如RJ45网口）混淆。
- **端口长度：** 端口号长度为 16 bit，能够表示 **65536** 的端口号 (0~65535)。



端口号的分类

分类名称	端口号范围	说明与用途
熟知端口号 (Well-known)	0 ~ 1023	指派给最重要的一些全球公认的TCP/IP应用进程。IANA负责分配。
登记端口号 (Registered)	1024 ~ 49151	供没有熟知端口号的应用程序使用。必须在IANA登记，以防重复。
动态端口号 (Dynamic/Private)	49152 ~ 65535	客户端进程运行时动态选择。通信结束后释放，可供其他客户进程复用。

应用层协议及端口号

应用层协议	传输层协议	默认端口号	功能简述
FTP (文件传输协议)	TCP	21 (控制), 20 (数据)	可靠的文件传输
SSH (安全外壳协议)	TCP	22	安全的远程登录
TELNET (远程终端)	TCP	23	远程登录协议
SMTP (简单邮件传输)	TCP	25	发送电子邮件
DNS (域名系统)	UDP (主要) / TCP	53	域名到IP地址解析
HTTP (超文本传输)	TCP	80	万维网数据传输

套接字 (Socket) 概念

什么是套接字？

在网络中采用发送方和接收方的套接字组合来识别端点。

所谓套接字，实际上是一个通信端点，它唯一地标识了网络中的一个具体进程。

公式： 套接字 = (主机IP地址 : 端口号)

全局唯一性

IP地址标识了网络中的唯一一台主机。

端口号标识了该主机上的唯一的一个应用进程。

因此，套接字 **Socket = (IP, Port)** 可以在全球互联网中唯一地确定一个通信端点。

1.3 无连接服务与面向连接服务

面向连接服务 (TCP)

通信双方在进行通信之前，必须先建立连接，在通信过程中，整个连接的情况一直被实时地监控和管理。

通信结束后，应该释放这个连接。

提供**可靠的**、全双工的、基于字节流的数据传输。

适用场景：文件传输、邮件、网页浏览。

无连接服务 (UDP)

通信双方不需要事先建立连接，直接发送数据。

发送方只管发送，不确认接收方是否收到，不保证数据的顺序和完整性。

提供**不可靠的**、尽最大努力交付的数据传输。

适用场景：实时视频会议、语音通话、DNS查询。

2.1 UDP协议概述

UDP 是在 IP 数据报服务之上增加了一些最基本的功能：**复用分用** 和 **差错检测**。

它是一个极其简单的协议，几乎没有对 IP 层增加太多额外开销。如果应用进程需要可靠性，则必须由应用层协议自己来提供可靠性保证（如应用层重传机制）。



UDP的主要特点（一）

-  **无连接：** 发送数据之前不需要建立连接，减少了开销和发送数据之前的时延。适合时间敏感的应用。
-  **尽最大努力交付：** 不保证可靠交付。主机不需要维持复杂的连接状态表。
-  **面向报文：** UDP对应用层交下来的报文，既不合并，也不拆分，而是 **保留这些报文的边界** 。发送方UDP对应用层交下来的报文，在添加首部后就向下交付IP层。

UDP的主要特点（二）

-  **无拥塞控制：** 网络出现拥塞不会使源主机的发送速率降低。这对某些实时应用很重要（如IP电话、实时视频直播），允许丢包但不允许太大的延迟。
-  **支持多种交互通信：** 支持一对一、一对多、多对一和多对多的交互通信（TCP只支持一对一）。
-  **首部开销小：** 只有 8 个字节，比 TCP 的 20 个字节首部要短得多。

UDP数据报首部格式

UDP 首部非常简单，仅包含 4 个字段，每个字段 2 字节（共 8 字节）。

字段名称	长度	说明
源端口 (Source Port)	2 字节 (16 bit)	需要对方回信时选用。不需要时可用全0。
目的端口 (Dest Port)	2 字节 (16 bit)	在终点交付报文时必须使用。若目的端口无进程，丢弃并发送ICMP。
长度 (Length)	2 字节 (16 bit)	UDP用户数据报的长度（首部 + 数据），最小值是8（仅有首部）。
校验和 (Checksum)	2 字节 (16 bit)	检测UDP用户数据报在传输中是否有错。有错就丢弃。可选字段（IPv4下）。

首部开销对比：TCP vs UDP



UDP 的头部固定为 8 字节，处理速度极快。而 TCP 为了保证可靠传输、流量控制和连接管理，固定首部至少需要 20 字节（还可以有选项字段，最多可达 60 字节）。

2.2 UDP校验机制与伪首部



校验范围： 与网络层IP数据报只校验首部不同，UDP的校验和 **把首部和数据部分一起都校验。**



伪首部概念： 在计算校验和时，要在UDP用户数据报之前增加 12 字节的伪首部。



伪首部的"伪"： 伪首部并不是UDP数据报真正的首部。只是在计算校验和时，临时和UDP数据报连接在一起，既不向下传递也不向上递交。



算法： 发送端在校验和字段全填0，把所有16位字相加（反码算术运算），将结果求反码，填入校验和字段。接收端连同伪首部一起按同样方法相加，无差错时结果应全为1。

UDP伪首部的结构 (12字节)

引入伪首部的目的是为了 UDP 校验不仅检查数据本身，还能验证数据报是否送达了正确的目的主机和端口。

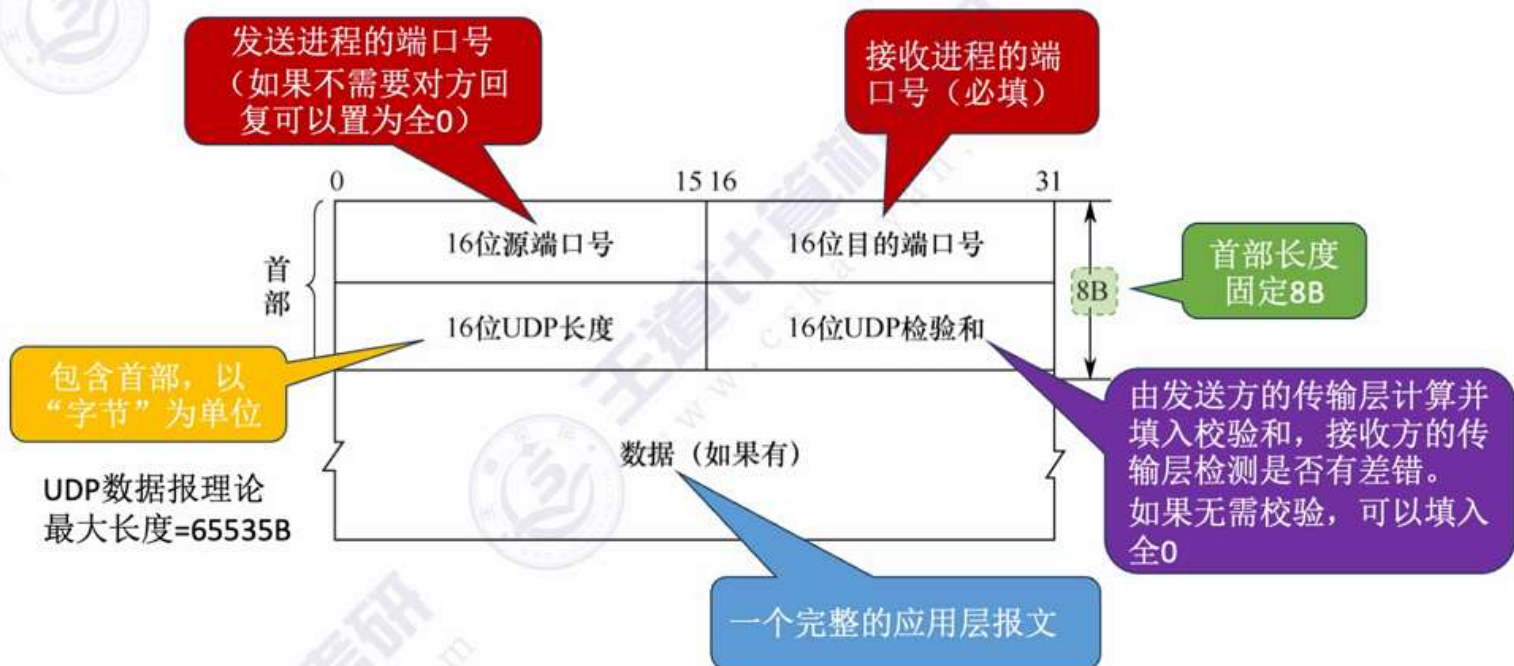
字段	长度	说明
源 IP 地址	4 字节	从 IP 首部提取，验证发送方身份。
目的 IP 地址	4 字节	从 IP 首部提取，验证是否误发给本机。
全 0	1 字节	填充字节。
协议字段	1 字节	封装UDP报文的IP数据报首部的协议字段 (UDP为17)。
UDP 长度	2 字节	UDP首部长度 + 数据部分长度。

UDP协议

与TCP的特点对比

- UDP首部很小，只占8B
 - UDP每次传输一个完整的报文，不支持报文自动拆分、重装
 - UDP是无连接的、不可靠的（可靠性可以交给应用层处理），也不支持拥塞控制
 - UDP支持一对一（封装成单播IP数据报）、一对多传输（封装成广播/多播IP数据报）
-
- TCP首部更大，占20~60B
 - TCP支持报文自动拆分、重装，因此可以传输长报文
 - TCP是有连接的、可靠的、支持拥塞控制
 - TCP仅支持一对一传输（因为通信双方的传输层必须先建立连接）

UDP数据报格式



3.0 TCP协议主要特点



面向连接

通信前必须三次握手建立连接，通信后必须四次挥手释放连接。只支持一对一（点对点）通信，不能广播或多播。



可靠交付

通过TCP连接传送的数据，无差错、不丢失、不重复，并且按序到达。



面向字节流

TCP把应用程序交下来的数据仅仅看成是一连串的无结构的字节流。
不保留应用层报文边界。

3.1 TCP报文段首部格式（核心字段）

字段	长度	功能说明 (408常考点)
序号 (seq)	4字节	本报文段所发送的数据的第一个字节的序号。保证按序到达。
确认号 (ack)	4字节	期望收到对方下一个报文段的第一个数据字节的序号。
数据偏移	4 bit	指出TCP报文段的数据起始处距离首部起始处有多远(以4字节为单位)。
窗口 (rwnd)	2字节	接收窗口，告诉发送方自己目前允许接收的数据量(流量控制)。
校验和	2字节	校验首部和数据，也要加上12字节的伪首部（协议号设为6）。

TCP首部控制位 (Flags)

连接建立与释放

SYN (Synchronize): SYN=1表示这是一个连接请求或连接接受报文。

FIN (Finish): FIN=1表示发送端的数据已发送完毕，要求释放传输连接。

RST (Reset): RST=1表示TCP连接中出现严重差错，必须释放连接后重新建立。

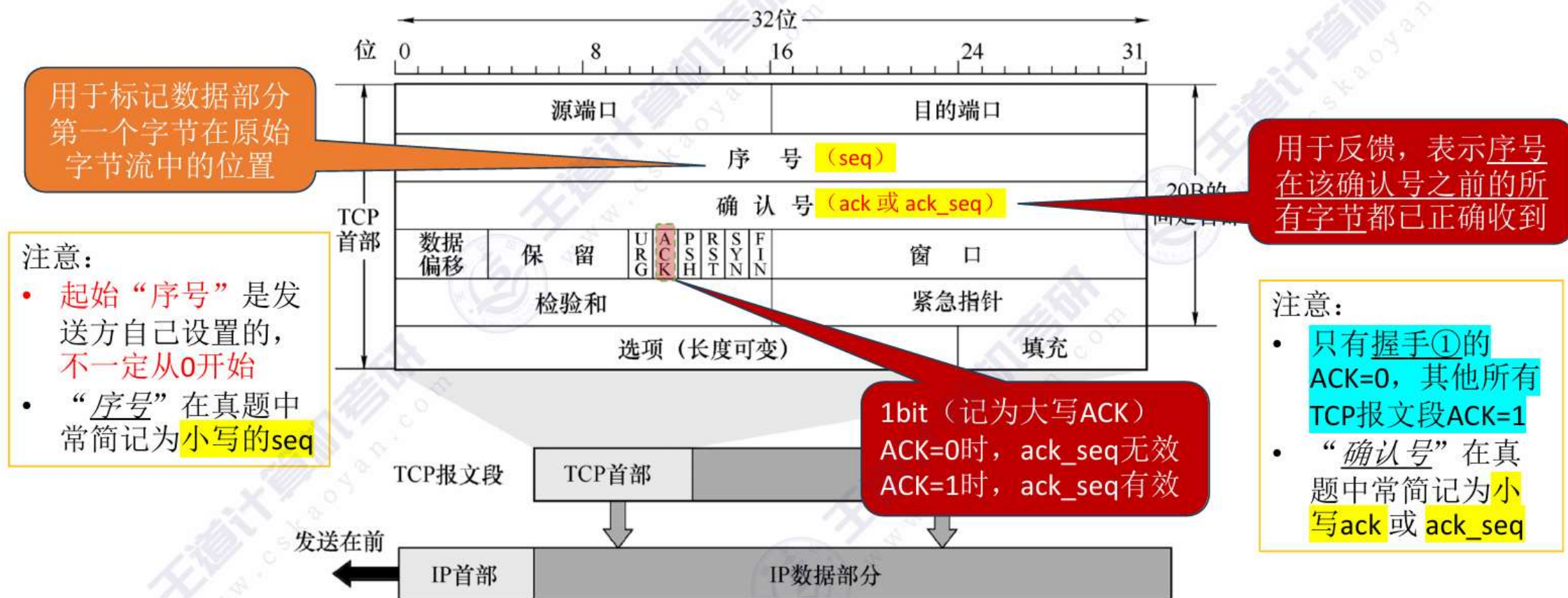
确认与推送

ACK (Acknowledgment): 仅当ACK=1时确认号字段才有效。TCP规定，连接建立后所有传送的报文段都必须把ACK置1。

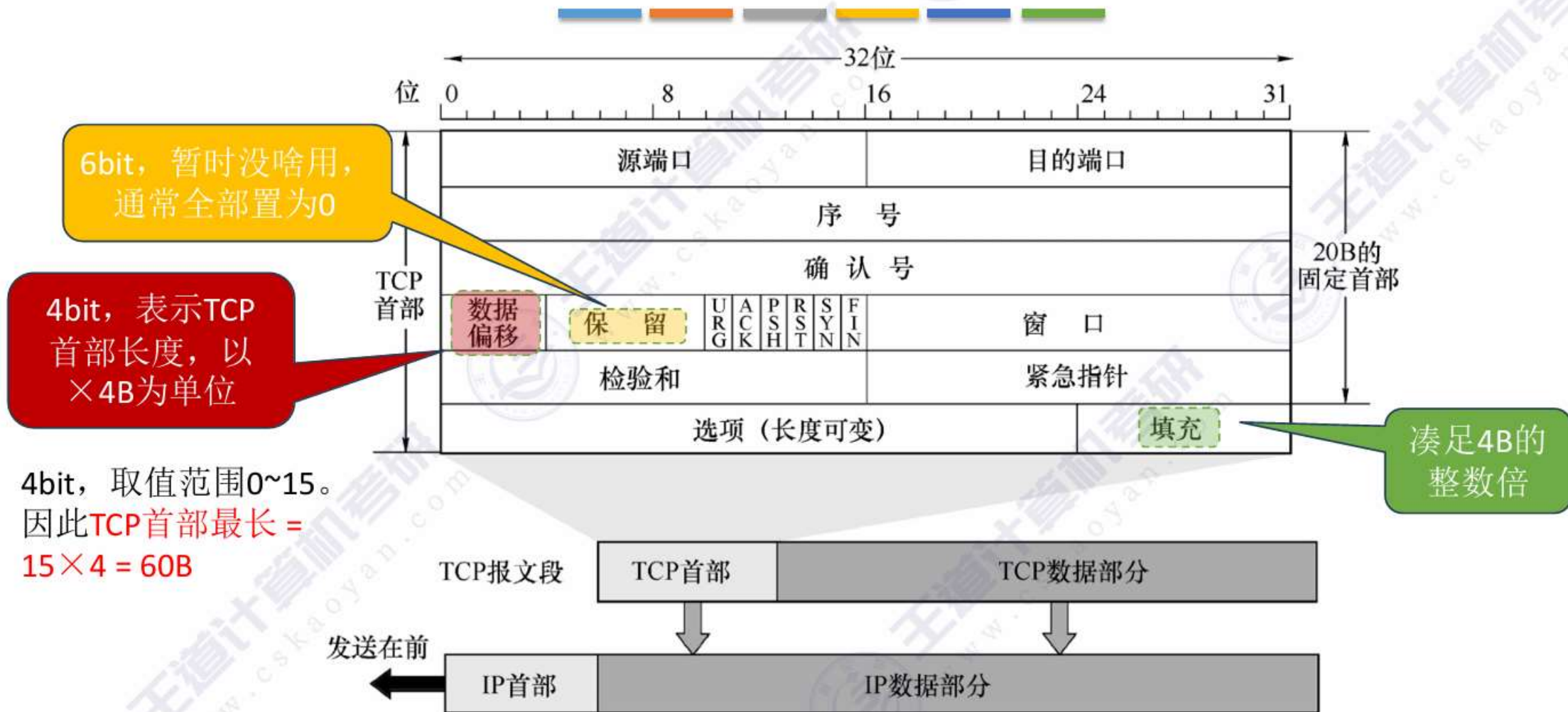
URG (Urgent): URG=1表明紧急指针字段有效，告诉系统此报文段中有紧急数据，应尽快传送。

PSH (Push): 接收方收到PSH=1的报文，应尽快交付接收应用进程，不再等到缓存填满。

TCP段格式（这页很重要、常考）



TCP段格式

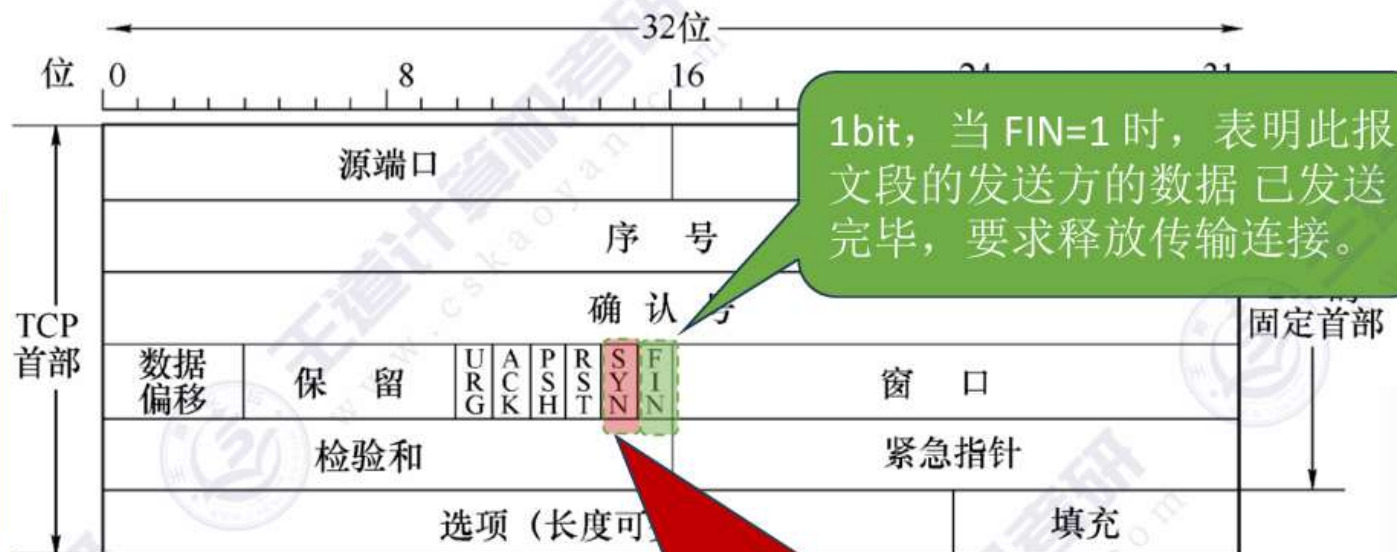


注1: *Data Offset* “数据偏移”这个翻译有迷惑性, 容易误解

注2: 在TCP首部中, 并不会专门记录TCP数据部分长度 (会根据IP首部、TCP首部的信息算出来)

TCP段格式（这页很重要、常考）

考题常见术语补充：
若**SYN=1**，可称为**SYN段**
若**FIN=1**，可称为**FIN段**
若**ACK=1**，可称为**ACK段**
其他标志位同理



synchronize

英 ['sɪŋkrənaɪz] 美 ['sɪŋ]

vt. (使) 同步；对准（钟，表）；

SYN: 同步位

FIN: 终止位 (Finish)

注意：

只有握手①、握手②的**SYN=1**，其他所有TCP报文段都是**SYN=0**

只有挥手①、挥手③的**FIN=1**，其他所有TCP报文段都是**FIN=0**

TCP段格式（这页很重要、常考）



receive

英 [rɪ'si:v] 美 [rɪ'si:v]

v. 得到, 收到; 遭受, 经 (某人为成员); 接收, 收

window

英 ['wɪndəʊ] 美 ['wɪndəʊ]

n. 窗, 窗玻璃; (计算机) 窗, 橱窗; 一丝的机会, 窗口; (物理) 大气窗; (

注意:

- 这个字段是实现“流量控制”的关键
- “窗口”在真题中常简记为rwnd或rcvwnd

TCP段格式 (MSS很重要、常考)



- 建立TCP连接时, 在握手①、握手②选项^②中协商MSS (Maximum Segment Size, 最大段长)
- MSS的值表示在接下来的数据传输中, 一个TCP报文段最多携带多少数据 (首部不算在内)
- 通常MSS不会设置太大, 以免在IP层被分片

三次握手 (Three-Way Handshake) 过程

第 1 次握手

Client 发送 SYN 报文:
SYN=1, seq=x
进入 **SYN-SENT** 状态。

初始状态

Server 处于 **LISTEN** 状态,
Client 处于 **CLOSED** 状态, 准备发起连接。

第 2 次握手

Server 收到后, 回复 SYN+ACK:
SYN=1, ACK=1, seq=y, ack=x+1
进入 **SYN-RCVD** 状态。

第 3 次握手

Client 收到回复, 发送 ACK:
ACK=1, seq=x+1, ack=y+1
双方进入 **ESTABLISHED** 状态。

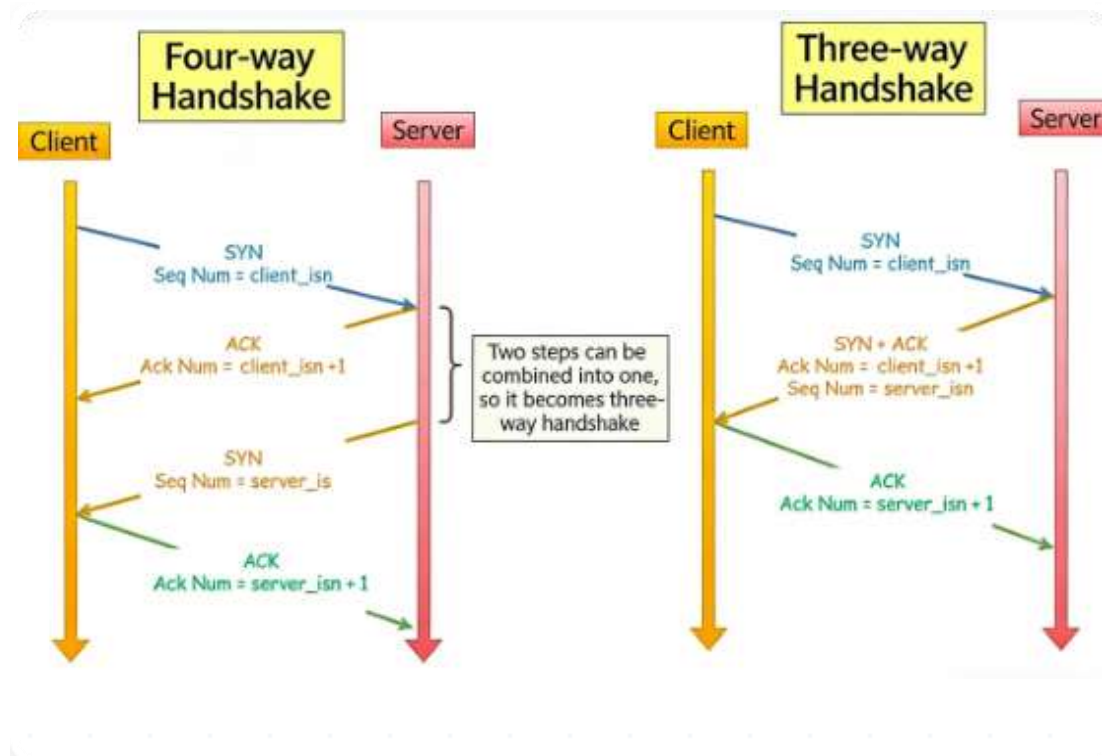
。

三次握手图解与核心问题

关键考点：为什么必须是三次握手？两次不行吗？

主要是为了防止“**已失效的连接请求报文段**”突然又传送到了服务端，因而产生错误。

如果使用两次握手：客户端发出SYN因为网络延迟没有到达，超时重发新的SYN，建立连接，传输完毕关闭。但此时那个延迟的旧SYN到达服务端，服务端以为客户端又要建立连接，发送ACK同意（两次握手即建立连接）。服务端会一直等待数据，造成资源白白浪费。



四次挥手 (Four-Way Teardown) 过程

第 2 次挥手

Server 回复 ACK:
ACK=1, seq=v, ack=u+1
Server 进入 **CLOSE-WAIT**,
Client 进入 **FIN-WAIT-2**。

第 4 次挥手

Client 回复 ACK:
ACK=1, seq=u+1, ack=w+1
进入 **TIME-WAIT** 等待 2MSL 后
CLOSED。

第 1 次挥手

Client 发送 FIN 报文:
FIN=1, seq=u
进入 **FIN-WAIT-1** 状态。

第 3 次挥手

Server 发送完数据，发 FIN:
FIN=1, ACK=1, seq=w, ack=u+1
进入 **LAST-ACK** 状态。

深刻理解 **TIME_WAIT** 状态

什么是 **MSL**?

MSL (Maximum Segment Lifetime) 是最长报文段寿命。TCP允许不同的实现设置不同的MSL值。

Client 发送最后一个 ACK 后，必须在这个状态停留 **2MSL** 时间，才能真正转变为 CLOSED 状态。

为什么要等待 **2MSL**?

1. **保证最后一个 ACK 到达**：如果 ACK 丢失，Server 会重传 FIN，Client 在 2MSL 内可以再次响应 ACK。
2. **防止“已失效报文”**：2MSL 足以让本次连接产生的所有报文段从网络中消失，使得下一个新的连接中不会出现旧连接的报文段。

实现可靠传输的四大机制



序号 (Sequence)

TCP将数据视为字节流，给每个字节编一个序号。报文段的序号字段是该段数据第一个字节的序号。



确认 (Acknowledgment)

采用**累积确认**机制。确认号是期望收到对方下一个报文段数据的一个字节的序号。



重传 (Retransmission)

分为**超时重传**（计时器到期）和**快速重传**（收到3个重复ACK）。

可靠传输

序号 —— 进程在建立连接时确定起始序号，数据传输过程中，每个字节对应一个序号

确认机制

累积确认规则 —— 如果收到 $\text{ack_seq} = n$ ，说明序号在 n 之前的所有字节都已正确接收

返回ACK的时机

推迟确认

推迟时间最多不能超过0.5秒（TCP标准规定）

如果自己也有数据要传送给对方，立即返回ACK段，并“捎带”自己的数据

若连续收到两个长度为MSS的报文段，就应该立即返回ACK段

两种ACK段

专门确认（咸鱼不规范术语） —— 一个ACK段只有TCP首部，而没有携带数据

捎带确认 —— 一个ACK段顺道携带了数据，就是“捎带确认”

重传机制

超时重传

每发出一个报文段，就设置一个计时器。若计时器到期还没收到确认，就重传这一报文段，并重置计时器

快重传
(冗余ACK)

配套机制：立即确认

作用：让可能出错的报文段尽早重传，而不是非要等到超时再重传

每收到一个TCP报文段，就立即返回一个ACK段

即使收到一个失序报文段，也要立即返回ACK段
(失序报文段会导致冗余ACK)

当发送方连续收到三个确认号相同的冗余ACK时，就立即重传该确认号对应的报文段

超时重传时间 (RTO) 的计算

TCP 采用自适应算法记录报文段发出的时间和收到确认的时间，这两个时间之差就是报文段的往返时间 RTT。TCP 会保留一个加权平均往返时间 SRTT。

$$\text{SRTT}_{\text{new}} = (1 - \alpha) \times \text{SRTT}_{\text{old}} + \alpha \times \text{RTT}_{\text{new}}$$

其中 α (平滑因子) 的典型值为 **1/8 (即 0.125)**。

超时重传时间 RTO 应该略大于 SRTT。公式为： **$\text{RTO} = \text{SRTT} + 4 \times \text{RTT_Dev}$** (RTT偏差的加权平均值)。

| 快速重传 (Fast Retransmit)

- ▶ **问题背景：** 超时重传需要等待计时器到期，这不仅增加了延迟，还会导致网络空闲。
- 3 **机制原理：** 接收方每收到一个失序的报文段后，就立即发出重复确认，而不是等待自己发送数据时捎带确认。
- 🚚 **触发条件：** 发送方只要一连收到 **3个重复确认**，就断定该确认号对应的报文段丢失，不必等待重传计时器到期，立即重传该报文。
- 📈 **优势：** 大大提高了整个网络的吞吐量。

3.4 TCP流量控制 (Flow Control)

什么是流量控制？

让发送方的发送速率不要太快，要让接收方来得及接收。

这是一种 **端到端** 的控制机制，由接收方通过TCP首部中的 **窗口字段 (rwnd)** 来实现。

发送方的发送窗口不能超过接收方给出的接收窗口大小。

死锁与持续计时器

若 B 向 A 发送了 $rwnd=0$ ，A 暂停发送。后来 B 有了缓存，发送非零窗口报文，若该报文丢失，A 会一直等，B 也会一直等（死锁）。

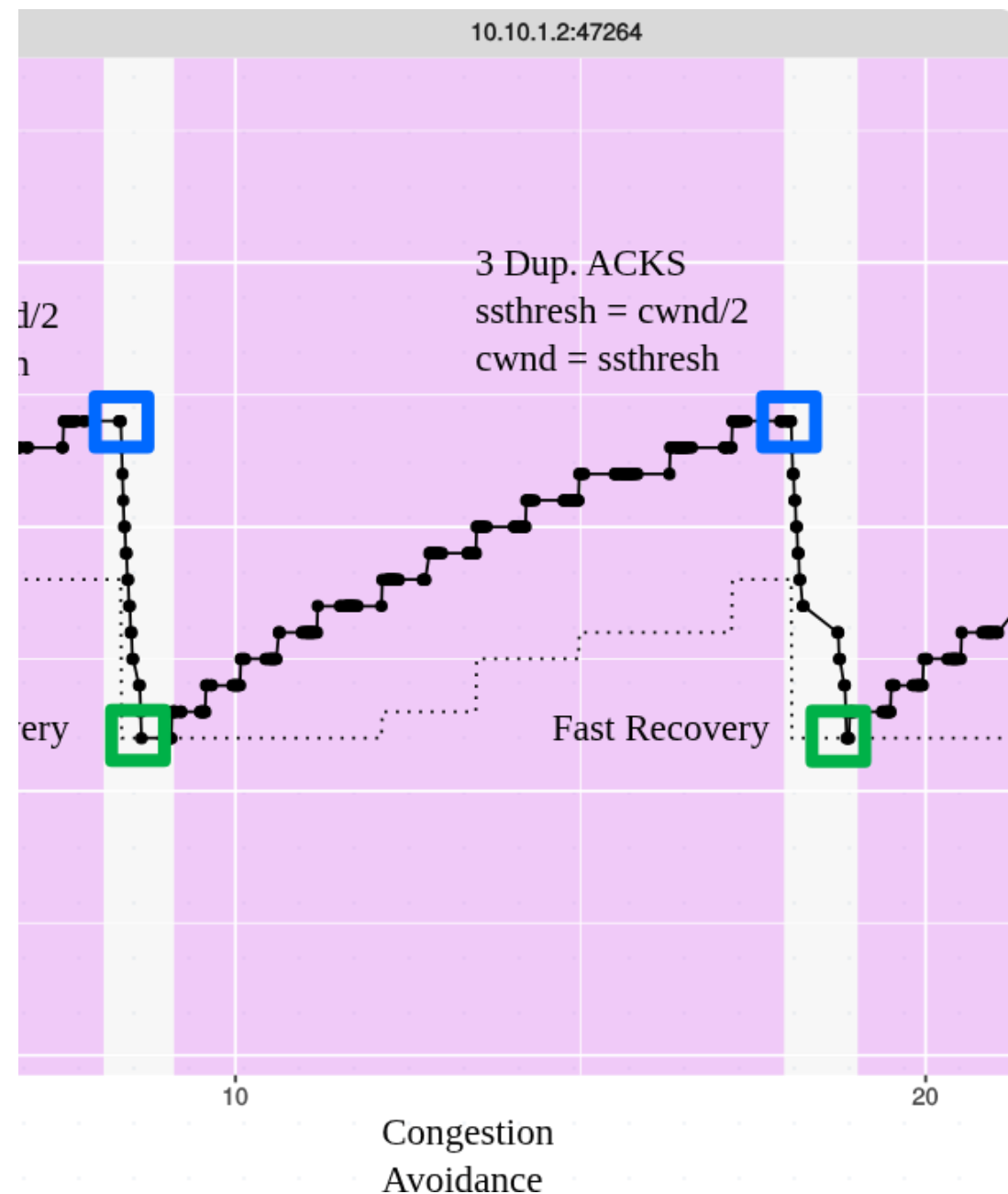
解决：TCP 为每个连接设有一个**持续计时器**。只要收到零窗口通知，就启动。到期后发送一个探测报文段。

3.5 TCP拥塞控制

流量控制 vs 拥塞控制:

流量控制是点对点的问题（接收方来不及收）；拥塞控制是全局性问题（网络中路由器/链路处理不过来）。

发送方的实际发送窗口大小 = $\text{Min}(\text{rwnd接收窗口}, \text{cwnd拥塞窗口})$ 。



拥塞控制四大算法



慢开始 (Slow Start)

从 $cwnd=1$ 开始，每经过一个 RTT， $cwnd$ 加倍 (指数增长: 1, 2, 4, 8...), 直到达到慢开始门限 $ssthresh$ 。



拥塞避免 (Cong. Avoidance)

当 $cwnd \geq ssthresh$ 时，每经过一个 RTT， $cwnd$ 加 1 (加法增大)，避免窗口增长过快引起拥塞。



快恢复 (Fast Recovery)

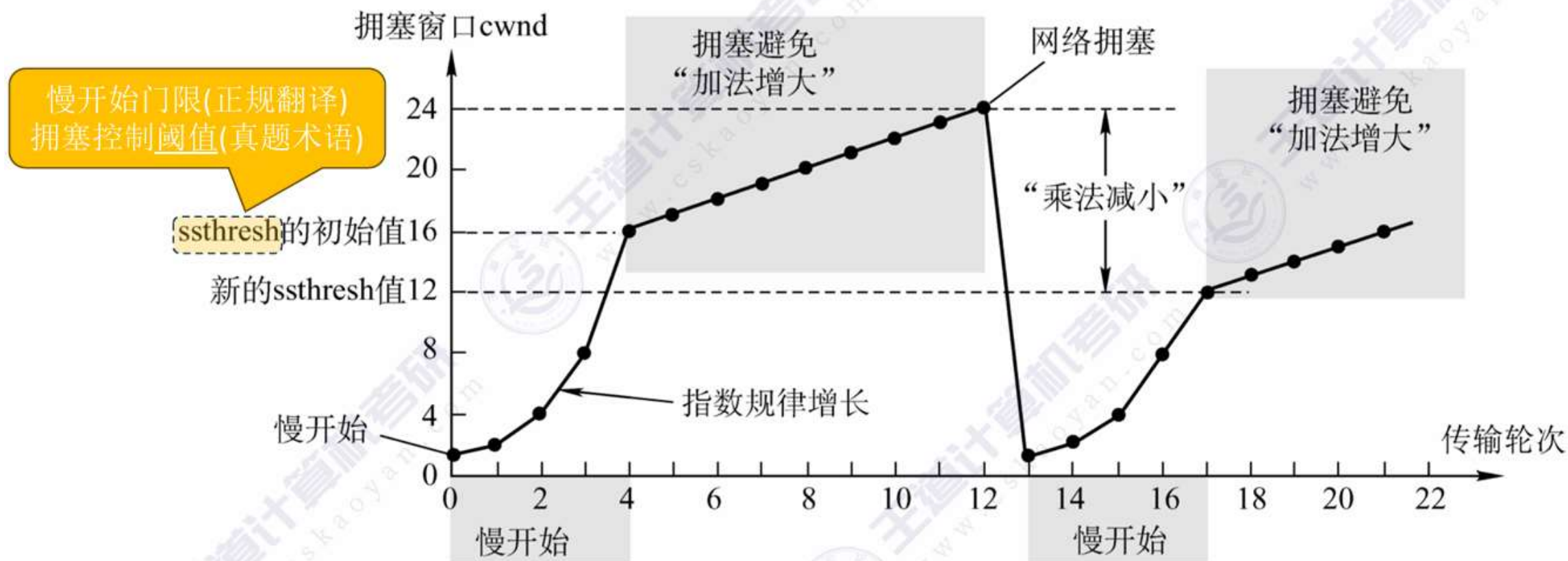
收到3个重复ACK时，不执行慢开始，而是将 $ssthresh = cwnd/2$ ， $cwnd = ssthresh$ ，然后执行拥塞避免。

拥塞窗口 (cwnd) 变化经典模型



红点处发生超时（网络拥塞），ssthresh减半，cwnd瞬间降至1（执行慢开始）。黄点处发生3个重复ACK，执行快恢复（cwnd降至新ssthresh并开始拥塞避免）。

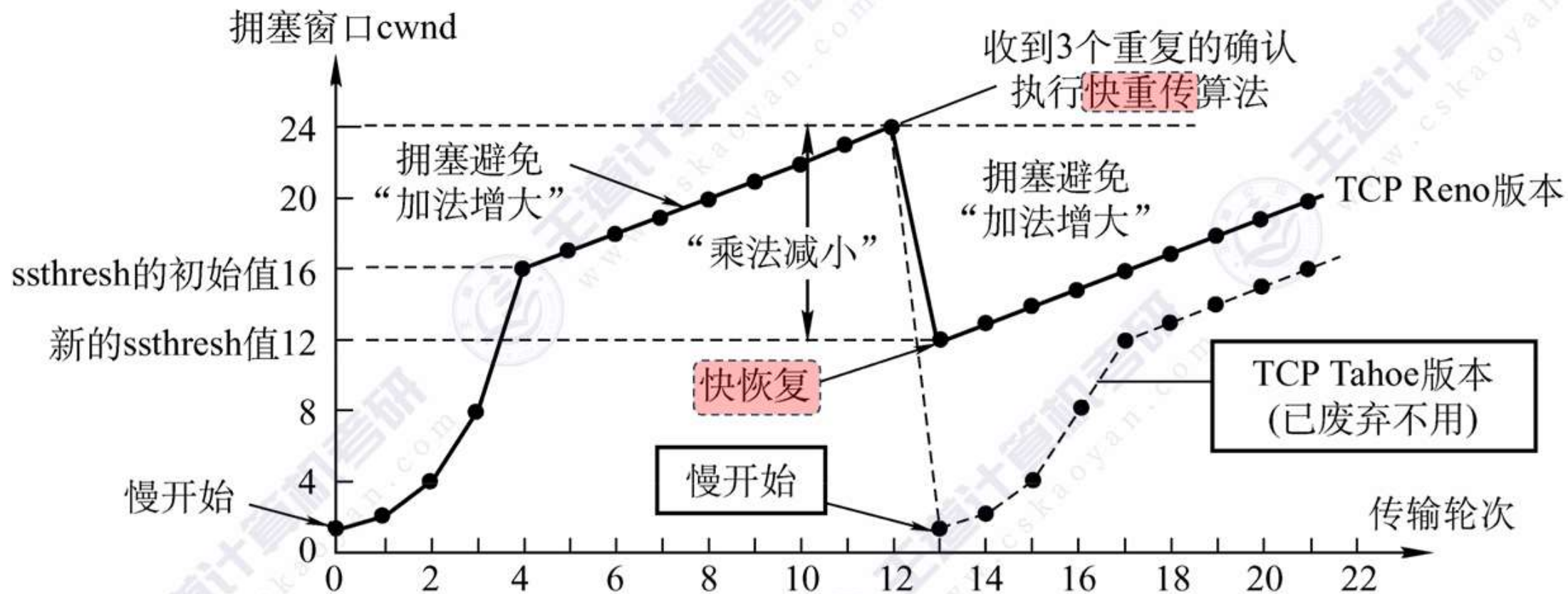
慢开始、拥塞避免



慢开始算法： cwnd 值从1开始，每收到一个ACK，就让 $cwnd+1$ (当 $cwnd < ssthresh$ 时适用)

拥塞避免算法： 在一个RTT内，即使收到多个ACK，也只能让 $cwnd+1$ (当 $cwnd \geq ssthresh$ 时适用)

快重传、快恢复



快重传：当发送方收到三个确认号相同的冗余ACK时，立即重传对应报文段

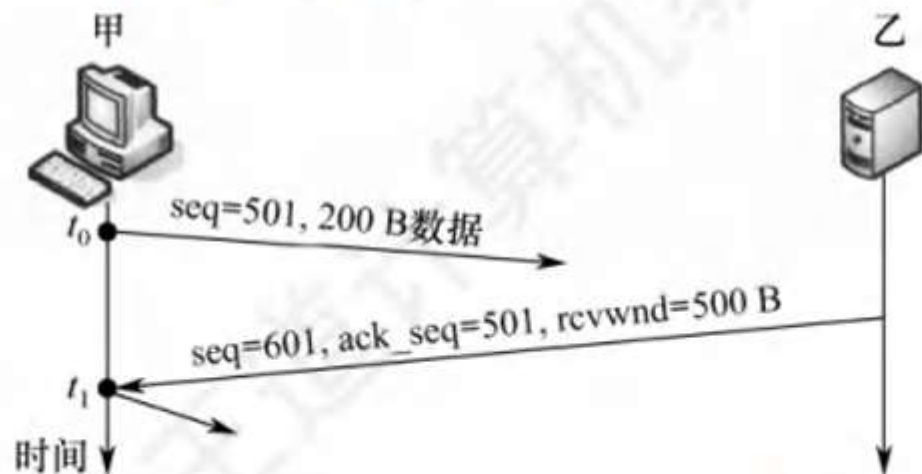
快恢复算法：一旦发生快重传，就将阈值、cwnd 都设为当前 cwnd 的一半，然后切换到为“拥塞避免算法”

14. 【2018 统考真题】UDP 协议实现分用时所依据的首部字段是 ()。
- A. 源端口号 B. 目的端口号 C. 长度 D. 检验和

14. B

传输层分用的定义是,接收方的传输层剥去报文首部后,能把这些数据正确交付到目的进程。C 和 D 显然不符。端口号用来标识主机中的应用进程。对于 A 和 B,源端口号在需要对方回信时选用,不需要时可用全 0。目的端口号在终点交付报文时使用,符合题意。

55. 【2021 统考真题】设主机甲通过 TCP 向主机乙发送数据，部分过程如下图所示。甲在 t_0 时刻发送一个序号 $\text{seq}=501$ 、封装 200B 数据的段，在 t_1 时刻收到乙发送的序号 $\text{seq}=601$ 、确认序号 $\text{ack_seq}=501$ 、接收窗口 $\text{rcvwnd}=500\text{B}$ 的段，则甲在未收到新的确认段之前，可以继续向乙发送的数据序号范围是（ ）。



A. 501 ~ 1000

B. 601 ~ 1100

C. 701 ~ 1000

D. 801 ~ 1100

55. C

主机甲发送 200B 数据的段后，继续发送数据的段的序号 $\text{seq}=701$ 。因为甲收到乙发来的确认序号为 501、接收窗口为 500 的段，即从序号 501 开始，甲累积还能发送 500B 的数据。因为甲此时已发送从序号 501 开始的 200B 数据，所以甲在未收到新的确认段之前，还能发送的数据字节数为 $500 - 200 = 300\text{B}$ ，还能发送的数据序号范围是 701~1000。

56. 【2022 统考真题】假设主机甲和主机乙已建立一个 TCP 连接，最大段长 $MSS = 1KB$ ，甲一直向乙发送数据，当甲的拥塞窗口为 $16KB$ 时，计时器发生了超时，则甲的拥塞窗口再次增长到 $16KB$ 所需要的时间至少是（ ）。

A. 4 RTT

B. 5 RTT

C. 11 RTT

D. 16 RTT

56. C

时刻 0 发生了超时，门限值 $ssthresh$ 变为拥塞窗口 $cwnd$ 的一半即 8，同时 $cwnd$ 置为 1，执行慢开始算法， $cwnd$ 指数增长，经过 3 个 RTT，增长到 $ssthresh$ 值；之后执行拥塞避免算法， $cwnd$ 线性增长，再经过 8 个 RTT，增长到 16，共花费 11 个 RTT，如下表所示。

时刻	0	1	2	3	4	5	6	7	8	9	10	11
拥塞窗口	1	2	4	8	9	10	11	12	13	14	15	16

57. 【2022 统考真题】假设客户 C 和服务端 S 已建立一个 TCP 连接，通信往返时间 $RTT=50ms$ ，最长报文段寿命 $MSL=800ms$ ，数据传输结束后，C 主动请求断开连接。若从 C 主动向 S 发出 FIN 段时刻算起，则 C 和 S 进入 CLOSED 状态所需的时间至少分别是（ ）。

A. 850 ms, 50 ms

B. 1650 ms, 50 ms

C. 850 ms, 75 ms

D. 1650 ms, 75 ms

57. D

TCP 连接的释放过程如书中图 5.8 所示。题目问的是最少时间，所以当服务器 S 收到客户 C 发送的 FIN 请求后不再发送数据，即服务器 S 同时发出确认 ACK 报文段和连接释放 FIN 报文段，忽略 FIN-WAIT-2 和 CLOSE-WAIT 状态。客户 C 收到服务器 S 发来的 FIN 报文段后，进入 CLOSED 状态还需等到 TIME-WAIT 结束，总用时至少为 $1RTT + 2MSL = 50 + 800 \times 2 = 1650ms$ 。服务器 S 进入 CLOSED 状态需要经过 3 次报文段的传输时间，即 $1.5RTT = 75ms$ 。

主机 A 与主机 B 连接在同一个以太网局域网内，双方建立了一条 TCP 连接。主机 A 向主机 B 发送一个包含 1000 字节应用层数据的 TCP 报文段。在物理介质传输过程中，由于电磁干扰，该帧的数据载荷部分发生了 1 个比特的翻转。假设不考虑其他网络异常，关于该数据包接收与处理的后续流程，描述最准确的是（ ）

- A. B 的物理层在收到比特流时通过纠错编码自动修复该错误，TCP 层无感知
- B. B 的数据链路层 MAC 子层通过 FCS 检测到错误并直接丢弃该帧，A 的 TCP 层最终会超时重传
- C. B 的网络层通过 IP 首部校验和检测到错误并丢弃该数据报，A 触发快速重传
- D. B 的传输层通过 TCP 校验和检测到错误，丢弃该报文段并向 A 发送 NACK 报文要求重传

【答案】：B

【解析】：

1. 以太网帧的尾部包含 4 字节的 FCS（帧检验序列），它保护的是整个以太网帧（包括目的地址、源地址、类型、数据载荷）。当 MAC 层收到帧并进行 CRC 校验时，一旦发现错误就会直接丢弃该帧
2. 既然帧在数据链路层已经被丢弃，它就根本没有机会被交给网络层，更不可能到达传输层。因此 IP 校验和（C 错）和 TCP 校验和（D 错）都不会被触发。TCP 协议没有 NACK 机制
3. 由于 B 收不到该报文，B 不会发送确认，最终 A 的 TCP 发送方会因为超时计时器溢出而重传该报文段。

主机 A 采用 UDP 协议和 TCP 协议，分别向主机 B 发送一段长度为 10 字节的极短应用层消息。两台主机处于同一个以太网段，且以太网帧的最小数据载荷限制为 46 字节，MAC 帧首部和尾部总开销为 18 字节。假设网络层均采用标准 20 字节的 IPv4 首部且无选项，传输层首部也无选项字段。请问，这两种协议最终在物理链路上实际发送的完整以太网帧总长度分别是多少字节？（ ）

- A. UDP 为 64 字节，TCP 为 68 字节
- B. UDP 为 68 字节，TCP 为 64 字节
- C. UDP 和 TCP 均为 64 字节
- D. UDP 和 TCP 均为 68 字节

【答案】：A

【解析】：

- 1. TCP 封装：10 字节数据 + 20 字节 TCP 首部 + 20 字节 IP 首部 = 50 字节的 IP 数据报。因为 $50 > 46$ （以太网最小载荷），所以不需要填充。完整帧长 = $50 + 18 = 68$ 字节**
- 2. UDP 封装：10 字节数据 + 8 字节 UDP 首部 + 20 字节 IP 首部 = 38 字节的 IP 数据报。因为 $38 < 46$ ，数据链路层必须在 IP 数据报尾部填充 8 字节的 0，使其达到 46 字节。完整帧长 = $46 + 18 = 64$ 字节**

主机 A 向外网的主机 B 发送一个总长度为 4000 字节的 UDP 数据报（包含 UDP 首部）。途经一个路由器时，由于出接口的 MTU 限制为 1500 字节，该 IP 数据报被路由器分片。以下关于该 UDP 数据报传输过程的说法中，正确的是（ ）

- A. 路由器在分片时，会将 UDP 首部复制到每一个 IP 分片中，以保证传输层信息的完整
- B. 路由器在分片后，会为每一个 IP 分片重新计算并更新 UDP 校验和
- C. 如果第二个 IP 分片在传输中丢失，主机 B 的 UDP 协议栈会要求主机 A 仅重发第二个分片
- D. 主机 B 的网络层必须将所有分片重新组装成完整的 IP 数据报后，才会交付给 UDP 层进行校验和验证

【答案】：D

【解析】：

1. A 错：IP 分片对传输层透明，UDP 首部（共 8 字节）仅存在于第一个 IP 分片的数据载荷中，后续分片不包含传输层首部
2. B 错：路由器只负责网络层转发和 IP 层首部的处理，根本不会去碰传输层的 UDP 校验和
3. C 错：IP 分片机制没有确认和重传功能，UDP 也是不可靠传输协议。如果一个分片丢失，整个 UDP 数据报重组失败被丢弃，不会请求重传局部
4. D 对：目的主机的 IP 层在超时时间内收到所有分片并成功重组后，才能将其作为一个完整的 UDP 数据报上交给传输层，此时 UDP 才会去验证校验和

客户端 C 和服务端 S 建立 TCP 连接并完成了双向的数据传输。现在客户端 C 应用程序主动调用 close() 关闭连接，向 S 发送了一个 FIN 报文段，S 收到后立刻回复了 ACK 报文段。此时，关于该 TCP 连接状态的描述，正确的是（ ）

- A. C 进入 CLOSE-WAIT 状态，S 进入 FIN-WAIT-1 状态
- B. C 进入 FIN-WAIT-2 状态，此时 C 仍能接收来自 S 的数据，但不能再向 S 发送应用层数据
- C. S 进入 LAST-ACK 状态，准备发送自己的 FIN 报文段
- D. C 进入 TIME-WAIT 状态，必须等待 2MSL 时间后才能彻底释放系统资源

【答案】：B

【解析】：

1. C 发送 FIN，C 进入 FIN-WAIT-1 状态
2. S 收到 FIN 并回复 ACK，S 进入 CLOSE-WAIT 状态。此时 C 收到该 ACK，C 进入 FIN-WAIT-2 状态
3. 在 FIN-WAIT-2 状态下，C 到 S 的方向已被关闭（即 C 不能发新数据），但 S 到 C 的方向仍然开放，S 依然可以将缓冲池中未发完的数据发送给 C，这就是 TCP 的半关闭（Half-close）特性，B 选项完全正确
4. 只有当 S 发送完自己最后的 FIN 后，S 才进入 LAST-ACK 状态；而 C 收到 S 的 FIN 并回复最后的 ACK 后，C 才进入 TIME-WAIT 状态

某高性能 Web 服务器的 IP 地址为 202.10.1.1，它在标准端口（TCP 80）上运行 HTTP 服务。现有成千上万个来自不同 IP 地址的客户端主机向该服务器发起 HTTP 长连接。关于操作系统的传输层是如何区分并管理这些数据流的，以下说法错误的是（ ）

- A. 尽管有上万个并发连接，该 Web 服务器依然只使用 80 这一个本地端口号来提供服务
- B. 当收到一个目标端口为 80 的 TCP 数据段时，服务器操作系统仅根据“源 IP 地址”和“源端口号”就能唯一确定该数据段属于哪个连接套接字
- C. 如果同一个客户端主机（单 IP）上的两个不同浏览器进程同时访问该服务器，它们必须被分配不同的源临时端口号
- D. HTTP 是面向连接的，所以传输层必须依靠由（源 IP、目标 IP、源端口、目标端口）构成的四元组来进行连接的多路分解

【答案】：B

【解析】：

1. B 选项错误。服务器操作系统在进行 TCP 连接的多路分解时，必须同时匹配完整的四元组（源 IP，源端口，目的 IP，目的端口），而不仅仅是源 IP 和源端口。虽然本题中目的 IP 和目的端口都是固定的（202.10.1.1:80），但从协议栈底层机制来说，缺一不可
2. A 对：服务端熟知端口不变，靠多线程/多进程关联不同的四元组套接字
3. C 对：同一个主机的不同进程访问同一外部服务，必须依靠不同的源临时端口号来区分
4. D 对：TCP 多路分解的本质就是四元组匹配

一个 TCP 连接采用慢开始、拥塞避免、快重传和快恢复算法（Reno 版本）。发送方的初始拥塞窗口 $cwnd = 1 \text{ MSS}$ ，慢开始门限 $ssthresh = 16 \text{ MSS}$ 。在第 1 轮、第 2 轮、第 3 轮和第 4 轮的传输中，所有报文段都按时收到确认。但在第 5 轮次发送的数据中，由于网络拥塞，发送方连续收到了 3 个针对同一报文段的重复 ACK。请问在执行完相应的拥塞控制算法后，发送方接下来要进入的拥塞控制阶段，以及此时的 $ssthresh$ 和 $cwnd$ 分别是多少？（ ）

- A. 进入慢开始阶段， $ssthresh = 8 \text{ MSS}$ ， $cwnd = 1 \text{ MSS}$
- B. 进入拥塞避免阶段， $ssthresh = 16 \text{ MSS}$ ， $cwnd = 8 \text{ MSS}$
- C. 进入拥塞避免阶段， $ssthresh = 8 \text{ MSS}$ ， $cwnd = 8 \text{ MSS}$
- D. 进入拥塞避免阶段， $ssthresh = 8 \text{ MSS}$ ， $cwnd = 11 \text{ MSS}$

【答案】：C

【解析】：

1. 推导拥塞窗口变化：

- 第 1 轮次： $cwnd = 1$
- 第 2 轮次： $cwnd = 2$
- 第 3 轮次： $cwnd = 4$
- 第 4 轮次： $cwnd = 8$
- 第 5 轮次： $cwnd = 16$ （此时刚刚达到 $ssthresh=16$ ）

2. 在第 5 轮次收到 3 个重复 ACK，触发快重传与快恢复

3. 快重传/快恢复规则：

- 新的 $ssthresh = \text{发生拥塞时 } cwnd \text{ 的一半} = 16 / 2 = 8 \text{ MSS}$
- 快恢复结束后， $cwnd$ 被设置为新的 $ssthresh$ 的值，即 $cwnd = 8 \text{ MSS}$ 。（注：执行快恢复算法的瞬间会加上 3，但在恢复完成后进入下一轮的稳态时， $cwnd=ssthresh$ ）
- 随后直接进入拥塞避免阶段（线性增长）。因此 C 正确

主机甲向主机乙发起 TCP 连接，甲发出的 SYN 报文段的序号 $seq=1000$ 。乙回复 SYN+ACK 报文段（序号 $seq=3000$ ）。随后，甲发送了三次握手的最后一个 ACK 报文段，并且在该报文段中“捎带”了 50 字节的 HTTP 请求数据。紧接着，甲又发送了第二个独立的数据报文段，包含了 100 字节的后续 HTTP 请求数据。请问，甲发送的这第二个独立数据报文段的 TCP 首部中，序号（seq）和确认号（ack）分别是多少？（假设期间乙没有向甲发送任何应用数据）

- A. $seq = 1001, ack = 3001$
- B. $seq = 1051, ack = 3001$
- C. $seq = 1051, ack = 3051$
- D. $seq = 1002, ack = 3002$

【答案】：B

【解析】：

1. 甲的第一步：SYN ($seq=1000$)，消耗一个序号
2. 乙的第二步：SYN+ACK ($seq=3000, ack=1001$)，消耗一个序号
3. 甲的第三步：ACK 且捎带数据。其序号等于乙期望收到的序号，即 $seq=1001$ 。确认号等于期望收到乙的下一个序号，即 $ack=3001$ 。由于该段携带了 50 字节数据，它将占据序号 1001 到 1050
4. 甲的第二份数据：紧跟在上一份数据之后，其起始序号应为上一个段的末尾加 1，即 $seq = 1001 + 50 = 1051$ 。因为在此期间乙没有发送任何数据，甲对乙的确认号保持不变，仍为 $ack = 3001$ 。

TCP 的可靠传输高度依赖于重传超时时间（RTO）的动态计算。在实际网络中，当发送方发生报文重传时，若收到该报文的 ACK，发送方会面临“确认二义性”问题（即无法分辨该 ACK 是对第一次发送的确认，还是对重传的确认）。此时 TCP 协议应采取的正确措施是（ ）

- A. 将 RTT 样本值取第一次发送和重传时间的平均值，重新计算 RTO
- B. 假设该 ACK 是对重传报文的确认，以此更新 RTT 估计值
- C. 忽略该重传报文段带来的 RTT 样本，不更新 RTT 估计值，并且将 RTO 计时器的值加倍
- D. 立刻触发快速恢复算法，将 RTO 计时器减半

【答案】：C

【解析】：

1. 在计算加权平均往返时间 RTTs 时，只要报文段被重传了，就绝对不采用其往返时间样本。这就直接排除了 A 和 B
2. 同时，为了应对网络可能真的发生了严重拥塞，报文段每重传一次，就把超时重传时间 RTO 增大一些（通常是加倍，即退避机制），直到通信恢复正常后再重新动态计算。因此 C 正确，D 的方向完全相反

不同的应用层协议根据其对数据可靠性、传输时延的不同需求，会选择不同的传输层协议（UDP 或 TCP）。下列哪一组应用层/网络层控制协议，它们在底层所使用的传输层协议是完全相同的？（ ）

- A. HTTP, FTP, TFTP
- B. SMTP, POP3, DNS
- C. DHCP, RIP, SNMP
- D. BGP, OSPF, ICMP

【答案】：C

【解析】：

1. A 错：HTTP 和 FTP 使用 TCP，而 TFTP（简单文件传输协议）使用 UDP
2. B 错：SMTP 和 POP3（电子邮件收发）必须保证可靠，使用 TCP；而标准的 DNS 查询使用 UDP
3. C 对：DHCP（动态主机配置）、RIP（路由信息协议，通过 520 端口）、SNMP（简单网络管理协议）全部基于 UDP 协议，因为它们更看重传输的实时性或机制的轻量化
4. D 错：BGP 使用 TCP（端口 179）；OSPF 直接封装在 IP 数据报中（协议号 89）；ICMP 同样直接封装在 IP 数据报中（协议号 1），它们不使用任何传统的传输层协议